



Riphah International University Islamabad

Project Proposal

Submitted by:

Syed Mujtaba Zaidi

SAP ID: 62081

Program: BS Cyber Security

BS (CY4-1)

Submitted to:

Sir Yawar Abbas Zaidi

Faculty of Computing (FC)

Subject: Operating System

Date: 27/02/2026

Project Title

Security Threat Analyzer. A Python-Based GUI Tool Using STRIDE & DREAD Threat Modeling Frameworks

1. Introduction

In today's digital world, cybersecurity threats are increasing rapidly. Organizations and individuals face serious risks from misconfigured systems, weak authentication, unencrypted data, and vulnerable services. Traditional security tools only collect data they do not analyze or explain the risk level of identified threats.

This project proposes a Security Threat Analyzer, a Python-based desktop application with a Graphical User Interface (GUI) that not only identifies security threats but also scores and prioritizes them using two internationally recognized threat modeling frameworks STRIDE and DREAD. The tool also integrates real-world vulnerability data through CVE Lookup and generates professional security reports.

2. Problem Statement

Most existing security tools such as Nmap, WHOIS, and basic scanners only perform data collection they show open ports or domain information but do not provide any threat analysis or risk scoring. Students and small organizations need a tool that can:

- 1) Identify what type of threat exists in a given system
- 2) Score how dangerous that threat is based on standard parameters
- 3) Match it with real-world known vulnerabilities
- 4) Generate a structured and professional security report

This project fills that gap by combining threat modeling with practical cybersecurity functionality in a single easy-to-use Python application.

3. Objectives

The main objectives of this project are:

- a) To design and develop a Python-based Security Threat Analyzer with GUI
- b) To implement the STRIDE model for automated threat identification
- c) To implement the DREAD model for risk scoring and prioritization
- d) To integrate CVE Lookup for real-world vulnerability matching
- e) To generate structured professional PDF security reports
- f) To apply Object-Oriented Programming (OOP) concepts throughout the project

4. Scope of the Project

The Security Threat Analyzer will consist of 5 core modules:

Module 1: GUI (Graphical User Interface)

- 1) Built using Python Tkinter library
- 2) User friendly input forms with checkboxes and dropdowns
- 3) Real time results display panel showing threats and scores
- 4) Files: main_window.py, input_panel.py, results_panel.py

Module 2: STRIDE Threat Analyzer

- 1) Accepts user input about system services, authentication, and encryption
- 2) Automatically identifies threats from all 6 STRIDE categories
- 3) Categories: Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege
- 4) Files: stride_analyzer.py, threat_rules.py

Module 3: DREAD Risk Scorer

- 1) Takes identified threats from STRIDE module as input
- 2) Scores each threat on 5 factors: Damage, Reproducibility, Exploitability, Affected Users, Discoverability (each on scale of 1-10)
- 3) Calculates overall risk level: Low / Medium / High / Critical
- 4) Files: dread_scorer.py, risk_calculator.py

Module 4: CVE Lookup Module

- 1) Searches publicly available CVE (Common Vulnerabilities and Exposures) database
- 2) Matches identified threats with real-world CVE entries
- 3) Displays CVE ID, description, and severity for each matched vulnerability
- 4) Files: cve_lookup.py, cve_parser.py

Module 5: Report Generator

- 1) Collects results from all modules
- 2) Generates a structured professional PDF security report
- 3) Saves reports locally in the outputs folder
- 4) Files: report_generator.py, pdf_exporter.py

5. Methodology

The development of the Security Threat Analyzer will follow a modular and object-oriented approach:

- a) User inputs system information through the GUI (services, authentication, encryption status)
- b) STRIDE module analyzes input and identifies threat categories
- c) DREAD module scores each identified threat on 5 parameters
- d) CVE module looks up real-world vulnerabilities related to identified threats
- e) All results are displayed on GUI in real-time
- f) Final results are exported as a structured PDF report
- g) OOP principles including classes, inheritance, and abstraction used throughout

6. Tools and Technologies

Component	Technology
Programming Language	Python 3
GUI Library	Tkinter
Programming Paradigm	Object-Oriented Programming (OOP)
CVE Database	NVD / NIST CVE API
Report Generation	ReportLab (PDF)
Platform	Windows / Linux
Total Source Files	12 Python Files

7. Project File Structure

```
security_threat_analyzer/
├── main.py                                <- Software entry point
├── gui/
│   ├── __init__.py
│   ├── main_window.py                    <- Main GUI window
│   ├── input_panel.py                    <- User input forms
│   └── results_panel.py                  <- Results display screen
├── modules/
│   ├── __init__.py
│   ├── stride/
│   │   ├── __init__.py
│   │   ├── stride_analyzer.py            <- STRIDE threat logic
│   │   └── threat_rules.py               <- Threat rules & conditions
│   ├── dread/
│   │   ├── __init__.py
│   │   ├── dread_scorer.py               <- DREAD scoring logic
│   │   └── risk_calculator.py            <- Low/Medium/High/Critical
│   ├── cve/
│   │   ├── __init__.py
│   │   ├── cve_lookup.py                 <- CVE API calls
│   │   └── cve_parser.py                 <- CVE data processing
│   └── report/
│       ├── __init__.py
│       ├── report_generator.py            <- Report logic
│       └── pdf_exporter.py               <- PDF export
├── data/
│   └── cve_cache.json                    <- Cached CVE data
├── outputs/
│   └── reports/                          <- Generated reports saved here
└── requirements.txt                      <- Python dependencies
```

8. Expected Outcomes

Upon successful completion, the project will:

- a) Deliver a fully functional Python desktop application with GUI
- b) Provide automated threat identification using STRIDE framework
- c) Provide risk scoring and prioritization using DREAD framework
- d) Match threats with real-world vulnerabilities via CVE database
- e) Generate professional and readable PDF security reports
- f) Demonstrate practical implementation of OOP and modular design in Python

9. Future Enhancements

Potential future improvements include:

- a) Integration of network scanning capabilities
- b) Database support for storing and comparing historical reports
- c) Web-based interface for remote access
- d) AI-based threat prediction and recommendation engine
- e) Multi-language report support

10. Conclusion

The Security Threat Analyzer combines academic programming knowledge with real-world cybersecurity methodology. By integrating STRIDE and DREAD threat modeling frameworks along with CVE lookup and report generation in a single Python GUI application, this project provides both educational value and practical cybersecurity relevance. The modular architecture ensures maintainability and scalability, making it a strong foundation for advanced security tool development in the future.